Assignment-Compiler Design

1. Define symbol table and its purpose in the process of compilation.
2. Define handle and handle pruning. Explain the stack implementation of shift reduce parser with the help of an example.
3. Give the translation scheme that converts infix expression into postfix and prefix notations. Generate the annotated parse tree for the string, **4*5+7-2,** under both the conversions.
4. Define operator grammar. Discuss the algorithm to compute Leading and Trailing for the Non-Terminals using a suitable example.

5. Differentiate among the parse tree, syntax tree, and directed acyclic graph using suitable examples.

6. Differentiate between the synthesized and inherited attributes with the help of appropriate examples.

7. Construct CLR parsing table for the following grammars:
   a) S → aAd | bBd | aBe | bAe
      A → c
      B → c

   b) S → ( )
      S → a
      S → (A)
      A → S
      A → A , S
8. Given the grammar below already augmented; construct the LR Parsing table and parse a suitable input of your choice.

   (0) S → *Stmts*
   (1) *Stmts* → *Stmt*
   (2) *Stmts* → *Stmts* ; *Stmt*
   (3) *Stmt* → *Var* = E
   (4) *Var* → id [E]
   (5) *Var* → id
   (6) E → id
   (7) E → (E)
9. Consider the following CFG G = (N={S, A, B, C, D}, T={a,b,c,d}, P, S) where the set of productions P is given below:

   S → A
   A → BC | DBC
   B → Bb | ε
   C → c | ε
   D → a | d

   a) Is this grammar suitable to be parsed using the recursive descendent parsing method? Justify and modify the grammar if needed.
   b) Compute the FIRST and FOLLOW set of non-terminal symbols of the grammar resulting from your answer in a)
   c) Construct the corresponding parsing table using the predictive parsing LL method.
   d) Show the stack contents, the input and the rules used during parsing for the input w = dbb

10. Given the following CFG grammar G = ({S,A,B}, S, {a, b, x), P) with P:

(1) S → A
(2) S → xb
(3) A → aAb
(4) A → B
(5) B → x

For this grammar answer the following questions:

a) Compute the set of LR(1) items for this grammar and the corresponding DFA. Do not forget to augment the grammar with the default initial production S' → S$ as the production (0).
b) Construct the corresponding LR parsing table.
c) Would this grammar be LR(0)? Why or why not? (Note: you do not need to construct the set of LR(0) items but rather look at the ones you have already made for LR(1) o answer this question.
e) Show the stack contents, the input and the rules used during parsing for the input string w = axb$
f) Would this grammar be suitable to be parsed using a top-down LL parsing method? Why?

11. Consider the following syntax-directed definition over the grammar defined by G = ({S, A, Sign}, S, {',', '-', '+', 'n'}, P) with P the set of production and the corresponding semantic rules depicted below. There is a special terminal symbol "n" that is lexically matched by any string of one numeric digit and whose value is the numeric value of its decimal representation. For the non-terminal symbols in G we have defined two attributes, *sign* and *value*. The non-terminal A has these two attributes whereas S only has the *value* attribute and Sign only has the *sign* attribute.

		S.val = A.val; A.sign = Sign.sign;
S → A Sign	||	print(A.val);
Sign → +	||	Sign.sign = 1
Sign → -	||	Sign.sign = 0
A → n	||	A.val = value(n)
A → A1 , n	||	A1.sign = A.sign; if(A.sign = 1) then A.va l = min (A_1.val, value(n)); else A.val = max(A_1.val,value(n));

For this Syntax-Directed Definition answer to the following questions:

a) Explain the overall operation of this syntax-directed definition and indicate (with a brief explanation) which of the attributes are either synthesized or inherited.
b) Give an attributed parse tree for the source string "5,2,3-" and evaluate the attributes in the attributed parse tree depicting the order in which the attributes need to be evaluated (if more than one order is

possible indicate one.)

c) Suggest a modified grammar and actions exclusively using synthesized attributes. Explain its basic operation.

12. What is activation record? Explain stack allocation of activation records using example.
13. Describe the symbol table format in detail.
14. Explain three loop optimization techniques with example.
15. Draw syntax tree and DAG for the statement ***x=(a+b)*(a+b+c)*(a+b+c+d)***
16. Draw a DAG for expression: ***a + a * (b – c) + (b – c) * d***

17. Obtain the flow graph for the following codes and perform the basic block optimization locally as well as globally.

a)

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        c[i][j] = 0.0;
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        for (k=0; k<n; k++)
            c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

b)

```
for (i=2; i<=n; i++)
    a[i] = TRUE;
count = 0;
s = sqrt(n);
for (i=2; i<=s; i++)
    if (a[i]) /* i has been found to be a prime */ {
        count++;
        for (j=2*i; j<=n; j = j+i)
            a[j] = FALSE; /* no multiple of i is a prime */
    }
```

18. For the following flow graph:
    a) Identify the loops of the flow graph.
    b) Statements (1) and (2) in B1 are both copy statements, in which a and b are given constant values. For which uses of a and b can we perform copy propagation and replace these uses of variables by uses of a constant? Do so, wherever possible.
    c) Identify any global common subexpressions for each loop.
    d) Identify any induction variables for each loop. Be sure to take into account any constants introduced in (b).

e) Identify any loop-invariant computations for each loop.